
Generating a Kubernetes Deployment Set from a Project-Intent Domain Model

Project Task 2: Model Transformation

Project Type: Code Generation

October 1, 2026

Joris Jonkers (S1829157)

Model Driven Engineering
University of Twente

Contents

1	Purpose of the Project	1
1.1	Goals	1
1.2	Role of the Transformation	1
1.3	Refinement of Tasks 0 and 1	1
1.4	Assumptions and Conditions	2
1.5	Scope	2
2	Alternatives and Justification	2
2.1	The Transformation Language	2
2.2	The Structure of the Transformation	3
3	Solution Design	4
3.1	Interface	4
3.2	Mappings and Queries	5
3.3	The Derivation Rules	5
3.4	The Trace and Late Assignment	8
3.5	The Black-Box Library	9
3.6	Stops	9
3.7	Running the Transformation	9
4	Tests and Evaluation	9
4.1	Testing Strategy	9
4.2	Results	10
4.3	Limitations	11
4.4	Reflection	11
A	Changes to the Task 1 Metamodels	12
A.1	The Pinned Inputs Metamodel	12
A.2	Changes to the Target Metamodel	12
A.3	Grammars for the Pinned Inputs	12
B	The Hand-in Archive	13

1 Purpose of the Project

1.1 Goals

The project generates the Kubernetes files of a small cluster from short deployment documents. Each application's repository holds one *project document*, which states what the application needs: its processes, their memory and CPU, the ports they offer, the services they depend on, and how the application is reached from outside. One *platform document* states what all applications share, such as the entry points of the cluster, its probe timings and its backup policies. Task 1 defined the two metamodels for this work in Ecore. The *source metamodel* describes both documents. The *target metamodel* describes the *resolved model*, which records every value the platform derives for an application, and the files that will be generated from it.

Task 2 implements the model transformation between the two. It takes the source models of every project in the cluster, the platform document, and three documents that other processes publish, and produces the resolved model of one project. The resolved model is the input of the code generation of Task 3. The transformation is where the derivation rules of the specification become executable: every value in a resolved model is computed by a rule in the transformation, and none is written by an author.

The transformation has two goals.

1. It derives the same decisions as the production implementation of the same specification, which is written in TypeScript and used to deploy the cluster. Both implementations are tested against the same committed result files, called *oracles*.
2. It stops with a message when it cannot derive a value. A resolved model with a missing or default value would generate files that look valid and deploy the wrong thing.

1.2 Role of the Transformation

Figure 1 shows the pipeline. Xtext parses each document into a source model, and OCL rejects invalid ones (Task 1). A first QVT-Operational transformation, the *lowering*, copies values that an author declared once for a whole project or application onto each process. Task 1 already contained the lowering, because the constraints are checked on its output. The second transformation, the *resolution*, is the subject of this report. It reads the lowered models and produces the resolved model. A small Java exporter then writes the resolved dependency edges as JSON, which is compared with the oracle. In Task 3, Acceleo templates will read the same resolved model and write the Kubernetes files.

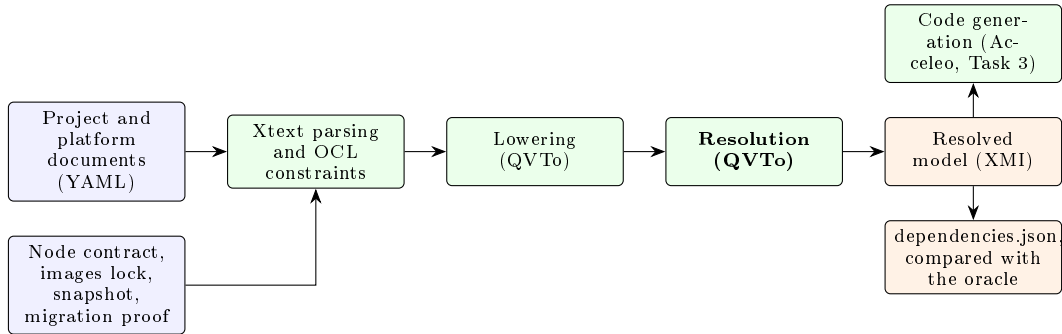


Figure 1: The Pipeline, With the Resolution This Report Describes

1.3 Refinement of Tasks 0 and 1

Task 1 planned one transformation from a source model, resolved against the platform document, to a resolved model. Writing it showed that the platform document is not enough. Three facts that the resolution needs come from other documents:

- the *node contract*, which lists the machines of the cluster and what each can hold;
- the *images lock*, which maps each image name in a project document to an immutable digest and the user the image runs as;
- the *cluster-state snapshot*, which records on which machine each stored volume already lives.

A project may also publish a *migration proof*, which records the database schema version its new release was tested against, and it may name configuration files, called *Assets*, whose text ends up in the cluster. The transformation therefore reads a third metamodel, the *pinned inputs*, beside the source metamodel. Appendix A describes this metamodel and the other changes to the Task 1 metamodels.

1.4 Assumptions and Conditions

- The input is valid. The OCL constraints of Task 1 reject an invalid source model before the transformation runs. A value that only a set of documents can check, such as an image missing from the lock, stops the transformation instead.
- The transformation resolves one project at a time, but reads all projects of the cluster, because a dependency on another project's service needs that service's address and port.
- The production implementation is the reference. Where the specification leaves a choice, the transformation makes the same choice, so that both produce the same oracles.
- The transformation runs without Eclipse, through the standalone QVTo executor, so that the build server runs it on every change. A launch configuration is also provided to run it in Eclipse (Appendix B).

1.5 Scope

The transformation covers every example project that the production implementation resolves: **minimal**, **auth**, **data**, **delivery**, **edge**, **observability** and **secrets**. Together these use every kind of value the specification derives: dependency edges, secrets, volumes with backups, configuration files, sidecar containers, database migrations, release gates and environment variables with placeholders.

Two things are outside the scope of Task 2. The eighth example, **knowledge**, is not yet resolved by the production implementation either, so it has no reliable oracle. The target metamodel also does not yet describe every file Task 3 must generate. The transformation produces the generated-file entries that the target metamodel can describe, and the rest are added with the templates in Task 3. Section 4.3 discusses both.

2 Alternatives and Justification

This section first compares the transformation languages the course offers (Section 2.1). It then compares the ways the transformation could be structured, and gives the chosen solution in general terms (Section 2.2). Section 3 describes the solution in detail.

2.1 The Transformation Language

The course offers ATL and QVT-Operational (QVTo). A third option is a transformation written in Java, by hand or with a small rule library such as SiTra. Table 1 compares the three against what this transformation has to do.

Requirement	ATL	QVTo	Java
Derive values from the whole cluster, not from one source element	helpers outside the rules	queries in the language	methods
Refer to a target object created by another rule	implicit resolution of matched rules	explicit resolveone through the trace	a trace kept by hand
Set a value only after the rest of the element exists	imperative do section	end section of a mapping	any order
Stop with a message on an undervivable value	no assertion statement	assert fatal	exception
Call code for an operation the language lacks	native library only	Java black-box library	not needed
Graded as a transformation language	yes	yes	no

Table 1: Transformation Languages Against the Requirements of the Resolution

Most values in a resolved model do not come from one source element. The eligible machines of a process depend on the node contract. A route's precedence depends on the other routes of the same exposure. The

ingress rules of a process depend on every process in the cluster that depends on it. ATL is strongest when one source element matches one target element, as in its matched rules. Here, most of the work would end up in helpers and called rules, which are imperative ATL without the matching. QVTo is imperative from the start, uses OCL for its expressions, and makes the trace an explicit operation. The transformation uses the trace in seven places, to point a gate member, a route or a generated file at the process the transformation created for it.

A Java transformation would be the easiest to debug. It would also keep the trace by hand, and it is not a transformation language, which the course grades. QVTo was therefore chosen. The one operation QVTo cannot express, computing a SHA-256 digest, is a Java black-box library (Section 3.5).

2.2 The Structure of the Transformation

Table 2 lists the structural alternatives that were considered, and the choice made for each.

Question	Alternatives	Choice and reason
One transformation or two	One transformation from the authored model; or a lowering first, then the resolution	Two. The constraints of Task 1 are checked on the lowered model, and the resolution reads one level of values instead of three.
How much to resolve per run	The whole cluster in one resolved model; or one project per run, with the other projects as input	One project per run. It matches the production implementation, which writes one resolved document per project, and keeps the model the templates read small.
Where the extra input documents come from	A Java YAML reader; hand-written XMI models; a metamodel with Xtext grammars	A metamodel with grammars. The same YAML files are read by both implementations, so their results can be compared.
How the inputs reach the transformation	One model extent per document; or one extent per metamodel	One extent for the intent (projects and platform) and one for the pinned inputs. The transformation selects documents by type.
How to compute digests	In OCL; in the Java pipeline before the run; in a black-box library	A black-box library. OCL cannot produce bytes, and a digest computed before the run could not cover values the transformation derives.
What to do with an underivable value	Write a default; write a diagnostic model; stop with a message	Stop. A default would produce files that deploy the wrong thing.

Table 2: Structural Alternatives and the Choices Made

2.2.1 A Lowering Before the Resolution

An author may declare a secret, a dependency or a memory limit once for a whole project, once for an application, or on a single process. The lowering merges these levels onto each process, so that the *effective* process contains everything that applies to it. Task 1 checks its constraints on this lowered model. If the resolution read the authored model instead, every rule that reads a process would have to merge three levels again, and could merge them differently from the constraints. Keeping two transformations means each rule of the resolution reads one place.

2.2.2 One Project per Run

The transformation runs once per project. The other projects are inputs: the resolution of **auth** reads the **data** project to find the address and port of the database it depends on. The alternative, one resolved model for the whole cluster, would also work, because QVTo can create several deployments in one run. It was rejected because the production implementation produces one resolved document per project and the oracles are per project, and because the templates of Task 3 generate one project's files at a time.

2.2.3 Grammars for the Pinned Inputs

The node contract, the images lock and the snapshot are YAML files in the same subset as the project documents. Three ways of reading them were compared. A Java YAML reader would be short, but it would add a parser the course does not teach, and it would read the files differently from the Xtext grammars that read the project documents. Hand-written XMI models of the three documents would be the quickest, but then the transformation would never read the files the production implementation reads, and the comparison between the two implementations would not cover them. The chosen solution adds one grammar per document. Each grammar inherits the terminals and indentation handling of the project grammar, so it adds only its own rules. The migration proof is read the same way. An Asset's file is the one input that is not parsed: its text belongs to the application, so the pipeline reads it as a string.

2.2.4 Stopping Instead of Defaulting

The specification states every derivation as total: each value either derives from declared input or the input is rejected. Some rejections need several documents, for example an image that the lock does not contain, and the constraints of Task 1 do not check those yet. The transformation therefore contains fatal assertions at each point where it would otherwise have to invent a value. Each assertion names what is missing. The same mechanism let the transformation be extended one example at a time: while an example was not yet supported, the transformation stopped on it with the name of the ticket that would add it.

3 Solution Design

The resolution is one QVTo transformation, `resolution.qvto`, of about 1000 lines. It contains 8 mapping operations and 86 queries. This section describes its interface (Section 3.1), how it is divided into mappings and queries (Section 3.2), the derivation rules one by one (Section 3.3), the use of the trace and of late assignment (Section 3.4), the black-box library (Section 3.5), the stops (Section 3.6), and how the transformation is run (Section 3.7).

3.1 Interface

Listing 1 shows the transformation's signature. It reads two model extents and writes one.

```
import dev.jorisjonkers.deploykit.emf.hashing;

modeltype PI uses 'https://jorisjonkers.dev/deploy-kit/project-intent/1';
modeltype IN uses 'https://jorisjonkers.dev/deploy-kit/pinned-inputs/1';
modeltype RD uses 'https://jorisjonkers.dev/deploy-kit/resolved-deployment/1';

transformation resolution(in intent : PI, in pinned : IN, out resolved : RD)
    access library Hashing;

configuration property project : String;
configuration property integrity : String;

property estate : Platform = intent.rootObjects()[Platform]->any(true);
property projects : Sequence(EffectiveProject) =
    intent.rootObjects()[EffectiveProject]->sortedBy(p | p.project)->asSequence();
property contract : NodeContract = pinned.rootObjects()[NodeContract]->any(true);
property lock : ImagesLock = pinned.rootObjects()[ImagesLock]->any(true);
```

Example 1: The Signature, Model Types and Inputs

The `intent` extent contains, for every project of the cluster, its authored model and its lowered model (`EffectiveProject`), and the platform document. The authored models stay in the extent because the platform document refers to applications in them: an entry point names the proxy application that serves it. Every rule reads the lowered models. The `pinned` extent contains the node contract, the images lock, the snapshot, each project's migration proof, and one `AssetFile` per configuration file. Two configuration properties give what is not derived: the project to resolve, and an identifier of the schema version, which the resolved model records as provenance. The transformation-level properties at the end of the listing select each input once, so the rules can refer to `estate`, `projects`, `contract` and `lock` by name.

The output extent receives one `ResolvedDeployment`. The entry point selects the project to resolve and maps it:

```

main() {
  projects->select(p | p.project = project)->any(true).map deployment();
}

```

Example 2: The Entry Point

3.2 Mappings and Queries

QVTo offers two ways to create a target object. A *mapping operation* records its source and result in the trace, so other rules can find the result later with `resolveone`, and calling it twice on the same source returns the same result. An *object expression* inside a query creates an object that nothing else needs to find. The transformation uses a mapping wherever another rule refers to the result, and an object expression otherwise. Table 3 lists the eight mappings.

Mapping	What it creates, and why it is a mapping
<code>EffectiveProject::deployment</code>	The <code>ResolvedDeployment</code> . Its <code>end</code> section sets values that need the complete model.
<code>EffectiveProject::unit</code>	The project's reconcile unit, the group Flux applies together. Each application refers to it.
<code>EffectiveProject::provenance</code>	The digests of every input, and the hash over them.
<code>EffectiveApplication::application</code>	A <code>ResolvedApplication</code> . Generated files refer to its exposures.
<code>EffectiveApplication::gate</code>	The release gate's inputs.
<code>EffectiveApplication::migrationOf</code>	The database migration of an application that manages its schema.
<code>Process::process</code>	A <code>ResolvedProcess</code> . Gate members, routes, policies and generated files refer to it.
<code>Exposure::exposure</code>	A <code>ResolvedExposure</code> . The generated route file refers to it.

Table 3: The Mapping Operations of the Resolution

The 86 queries are of four kinds. Value queries derive one value, such as a deadline or an address. Object queries build a contained object, such as a probe or a backup plan. Spelling queries turn an enumeration literal of the source metamodel into the string the target metamodel records, for example `AlertClass::businessHours` into `business-hours`. Stop queries, whose names start with `un`, raise a fatal assertion (Section 3.6).

Figure 2 shows the order in which the mappings run. The `deployment` mapping creates the reconcile units first, then the applications, the provenance and the generated files. Each application maps its processes before its exposures and its gate, because those refer to the processes.

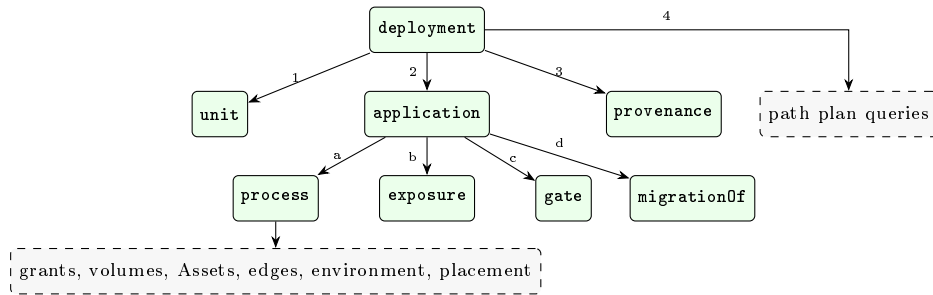


Figure 2: Mapping Invocations, in the Order They Run (Dashed: Groups of Queries)

3.3 The Derivation Rules

Each rule implements a derivation of the specification. Table 4 lists them by the target element they produce, with the rule that computes each and the source of its input. The rules are written as the specification states them; where it leaves a choice, they follow the production implementation.

Table 4: Derivation Rules, by Target Element

Target value	Rule	Derivation
<i>Application</i>		
namespace	<code>namespaceOf</code>	The project’s name followed by <code>-system</code> .
alert class	<code>AlertClass::spelling</code>	Copied from the application’s observability declaration.
reconcile unit and its order	<code>unit, afterOf</code>	One unit per project, <code>apps-<project></code> . It follows the unit of every other project its edges reach, and the secrets unit if any process holds a secret.
secret store	<code>synced, storeEndpoint</code>	The address of the secret store’s <code>http</code> port, recorded only if a secret or a backup credential is copied into the cluster.
migration	<code>migrationOf</code>	The migration image from the lock entry <code><application>-migration</code> , and the tested schema version from the project’s migration proof.
release gate	<code>gate, switchesBlueGreen</code>	The inputs of the service that holds a new version back until all its processes are healthy. One member per blue/green process that has a readiness probe. The deadline is the slowest member’s. The analysis cadence comes from the platform.
<i>Process</i>		
image, user and group	<code>locked</code>	The lock entry for the image name: repository, digest, user and group.
switchover	<code>switchoverOf</code>	<code>stop-start</code> for an interrupted process, <code>rolling</code> for the platform’s delivery tools, <code>blue-green</code> otherwise.
deadline	<code>deadlineOf</code>	Three times the startup budget, rounded down; the budget itself, rounded up, for a process that runs once before a release; 600 seconds without a budget.
probes	<code>Probe::cadenced,</code> <code>Probe::target</code>	The declared target with the platform’s period, timeout and failure count. The startup probe polls the liveness target every five seconds, for as many attempts as the startup budget allows.
eligible machines	<code>eligible,</code> <code>requestedMemory</code>	Every machine of the node contract whose memory and CPU fit the process and its sidecars, twice for a continuous process because its old and new versions run side by side during a switch, and that matches every declared architecture, site, capability, GPU and disk medium.
bound machine	<code>boundOf</code>	The machine the snapshot records for one of the process’s volumes.
environment	<code>environmentOf,</code> <code>Placeholder::entryFor</code>	The base env file, overlaid by the cluster’s own file. Each placeholder is replaced by a secret reference, an edge’s host or port, an exposure’s URL, or the process’s own name or namespace. The runtime profile adds the telemetry variables and the port. Sorted by name.
secrets	<code>Grant::resolvedFor</code>	A key-value grant keeps its path and keys and gets the name of the Kubernetes Secret it is copied to. A transit or database grant becomes the Vault paths its policy covers.

Target value	Rule	Derivation
volumes and backups	Volume::resolvedFor, backupFor	A recoverable or irreplaceable volume gets a backup plan: the platform's schedule and retention for its class, the backup image for the process's database engine, and a separate identity and claim for the backup.
configuration files	Asset::resolvedFor	The file's text, under a name made of the process, the file name and the first ten hex digits of the text's digest, so a changed file gets a new name.
dependency edges	edgeOf	The first process in the cluster whose application and port the edge names, with its address and the network peer it admits. Otherwise the platform's register of external providers.
<i>Exposure</i>		
entry point	tierFor	The first entry point of the platform that carries the exposure's audience.
route precedence	rankOf, precedenceOf	Exact routes before prefix routes, longer prefixes before shorter.
middleware	middlewareOf	Authentication for an authenticated route, then security headers, then a redirect if declared.
<i>Network policy, per process</i>		
ingress	ingressOf, consumersOf	The proxy of each route's entry point, the metrics collector if the process is scraped, and every process in the cluster with an edge to one of its ports. Each peer once.
egress	egressOf, storePeers	The cluster's DNS, and the secret store if the process holds a secret.
<i>Deployment</i>		
provenance	provenance	One digest per input, and a hash over all of them and the schema version.
revision	end section of deployment	The digest of each application's complete element.
generated files	projectFiles, applicationFiles, edgeFiles	The path of every file the target metamodel describes, with the processes or exposures it is generated from (Section 3.3.4).

3.3.1 A Process

Listing 3 shows the start of the process mapping. The `init` section finds the lock entry and the declared probes once, so the assignments below can use them. Each assignment calls the query of Table 4 that derives it.

```

mapping PI::Process::process(in app : EffectiveApplication, in prj : EffectiveProject)
: ResolvedProcess {
  init {
    var own : LockedImage := locked(self.image);
    var declared : Probes := if self.probes <> null and self.probes.oclIsKindOf(Probes)
      then self.probes.oclAsType(Probes) else null endif;
  }
  name := self.name;
  identity := self.name;
  image := own.repository + '@' + own.digest;
  uid := own.uid;
  gid := own.gid;
  if self.lifecycle = Lifecycle::application then switchover := switchoverOf(app, self) endif;
  deadline := inSeconds(deadlineOf(self));
  secrets := self.secrets->collect(g | g.resolvedFor(self));
  identityToken := self.secrets->exists(g | g.delivery = Delivery::selfDelivery);
  placement := object ResolvedPlacement {
    eligibleNodes := contract.nodes->select(n | eligible(self, n))->collect(n | n.name);
  };
  dependencies := self.dependsOn->collect(e | edgeOf(e));
  environment := environmentOf(self, prj)->sortedBy(e | e.name);
}

```

Example 3: The Process Mapping, Abridged

3.3.2 A Dependency Edge

A dependency edge names an application and a port name, for example `platform-postgres` and `postgres`. The rule in Listing 4 looks for the first process in the cluster that belongs to that application and offers that port. The resolved edge records the process's in-cluster address and the network peer that the consumer may reach. If no process offers the port, the platform's register of external providers is tried, which is how `auth` reaches the mail server outside the cluster. If neither has it, the run stops.

```
query edgeOf(e : DependencyEdge) : ResolvedEdge =
  let providers : Sequence(Tuple(project : String, process : PI::Process)) =
    projects->collect(prj | prj.applications->select(a | a.id = e.application).processes
      ->select(p | p.provides->exists(s | s.key = e.surface))
      ->collect(p | Tuple{project = prj.project, process = p}))>flatten() in
  if providers->notEmpty() then
    let first : Tuple(project : String, process : PI::Process) = providers->first() in
    let at : Integer = first.process.provides->select(s | s.key = e.surface)->any(true).value in
    object ResolvedEdge {
      application := e.application;
      surface := e.surface;
      address := first.project.name + '.' + namespaceOf(first.project)
        + '.svc.cluster.local:' + at.toString();
      peers := Sequence(object PolicyPeer {
        namespace := namespaceOf(first.project); process := first.process.name; port := at; });
    }
  else ... the platform's register, or unprovided(e) ... endif;
```

Example 4: Resolving a Dependency Edge, Abridged

3.3.3 Environment Variables and Placeholders

An env file may contain placeholders, such as `DB_HOST=${dependency:platform-postgres.host}`. The query `Placeholder::entryFor` handles the four kinds. A secret placeholder becomes a reference to a key of a secret the process holds, so the value never appears in a file. A dependency placeholder takes the host or the port of the process's own edge to that application, from `edgeOf`. An exposure placeholder takes the URL, host or scheme of an exposure anywhere in the cluster. An identity placeholder takes the process's own name or namespace. Text after the placeholder is kept, as in `AUTH_LOGIN_URL=${exposure:auth.public#url}/login`, except after a secret. A placeholder that names nothing stops the run.

3.3.4 The Path Plan

The *path plan* assigns each generated file its path. The project gets a namespace file, an index of its directories and a default-deny network policy. Each application gets a directory with one file per kind of resource: the workloads, the blue/green switch definitions, the service accounts, the metrics definition if it is scraped, and the network policies. A file lists every process it applies to, and the policy file contains one policy per process. Each exposure gets a route file in the directory of its entry point, and each entry point gets an index of its route files. Task 3's templates generate each file from its entry in the model, so the templates do not decide any path.

3.4 The Trace and Late Assignment

The transformation uses the trace for every reference from one created element to another. A gate member, a route and a metrics file each refer to a `ResolvedProcess`, and the route file refers to a `ResolvedExposure`. Each of these uses `resolveone` on the source element, for example `p.resolveone(ResolvedProcess)`. This works because the application mapping maps its processes first.

Three values can only be computed once the rest of the model exists, and they are set in `end` sections:

- each application's *revision*, the digest of its complete element, which names this release of it;
- the provenance's render hash, which covers every input digest;
- the order between reconcile units, which refers to units the same mapping created.

```
end {
  result.reconcileUnits->first().after :=
    result.reconcileUnits->excluding(result.reconcileUnits->first());
  result.applications->forEach(a) {
```

```

    a.revision := a.oclAsType(ECORE::EObject).revisionOf() };
}

```

Example 5: The End Section of the Deployment Mapping

3.5 The Black-Box Library

A digest is a SHA-256 hash over a canonical text form of a model element. OCL has no operation that produces bytes or hashes them. The class `Hashing` therefore provides three contextual operations: `digestOf` for any model element, `textDigestOf` for a string, and `revisionOf` for a resolved application. Each walks the element reflectively, writes canonical JSON and hashes it. The transformation calls no other Java code. The pipeline registers the library with the standalone executor before the run, and in Eclipse a `plugin.xml` registers it with the same unit name.

Two details of the QVTo implementation shaped the library. An operation that takes a plain `EObject` is not found when it is called on a model element, because a model type does not conform to the `Ecore` type in the compiler. Each call therefore casts first: `p.oclAsType(ECORE::EObject).digestOf()`. And the unit name could not be the package name of the class, because `resolve` is a keyword of QVTo, so the unit is registered as `dev.jorisjonkers.deploykit.emf.hashing`.

3.6 Stops

Fourteen fatal assertions stop the run when a value cannot be derived. Each is in a stop query, which the rule calls instead of returning an invalid value:

```

query unlocked(alias : String) : LockedImage {
    assert fatal (false) with log(alias + ' is not locked: E_UNLOCKED_IMAGE');
    return null;
}

```

Example 6: A Stop Query

The stops cover an image missing from the lock, an edge that no process or provider serves, a placeholder that names nothing, a variable that an author wrote although the runtime profile sets it, an `Asset` whose file is missing or contains a placeholder, a backed-up volume without a schedule or a database engine, a platform without a secret store or delivery policy, and an application whose processes would need two different workload kinds in one file. Where the specification defines an error code for the situation, the message contains it.

3.7 Running the Transformation

The class `Resolution` runs the transformation through the standalone `TransformationExecutor`. It registers the black-box library, sets the two configuration properties, passes the two extents, and returns the `ResolvedDeployment`, or fails with the executor’s diagnostic. The pipeline class `Pipeline.resolve` builds the extents: it parses each file with the grammar its name selects, checks the constraints, adds the env files, lowers each project and reads each `Asset`’s file. For each example, the build writes the dependency edges, the resolved model and both input extents as files under `target/parity/<example>/`. The launch configuration `resolution.launch` runs the transformation on those extents in Eclipse.

4 Tests and Evaluation

4.1 Testing Strategy

The transformation is tested at four levels. Each level answers a different question, and the build runs all four on every change.

1. **Parity on the dependency edges.** For each example, the pipeline resolves the project and exports the resolved dependency edges as canonical JSON. A parity test compares the file byte for byte with the committed oracle `expected/dependencies.json`, which the production implementation is tested against as well. The edges are the part of a resolved model that both implementations must agree on before either generates files. The parity test reads only the files the pipeline wrote, so it tests the pipeline as a user runs it.

2. **Equality with a hand-written target model.** Task 1 included a target model of `minimal`, written by hand from the specification before the transformation existed. A test compares the transformation’s output with it using EMF’s structural equality, `EcoreUtil.equals`. This covers every value of the model, the generated-file entries included.
3. **A cross-check against the production implementation.** The production implementation writes a resolved document per application, `expected/resolved.json`. Its shape differs from the target meta-model, so it is not an oracle for this implementation. During development, a script compared every value that both forms record (Section 4.2.1).
4. **Tests of the transformation’s own interface.** JUnit tests run the transformation on models built in code: the smallest project that resolves, a project the input does not contain, and an application whose processes need two kinds of workload, which must stop the run. Other tests check that each grammar of the pinned inputs reads its document, and that a rejected input set is never resolved.

The Java code around the transformation (the pipeline, the runner, the black-box library and the exporter) is held to 100% line and branch coverage, and to a mutation score of 100%: every change a mutation tool makes to it must fail a test. The QVTo code is not measured by these tools; the first three levels test it.

4.2 Results

Table 5 lists the seven examples. All seven resolve, and for each the exported edges equal the oracle. The output of `minimal` equals the hand-written model.

Example	Apps	Proc.	Edges	Secrets	Volumes	Env.	Edges equal oracle
<code>minimal</code>	1	1	0	0	0	6	yes
<code>auth</code>	1	2	4	3	0	21	yes
<code>data</code>	3	3	0	1	3	3	yes
<code>delivery</code>	2	2	0	0	0	4	yes
<code>edge</code>	2	2	0	0	0	0	yes
<code>observability</code>	2	2	0	0	0	0	yes
<code>secrets</code>	1	1	0	0	0	0	yes

Table 5: The Resolved Examples. Columns Count Applications, Processes, Dependency Edges, Secrets, Volumes and Environment Variables in Each Resolved Model.

The examples are small, but together they use every rule of Table 4. `auth` has two processes switched together behind one release gate, four edges (three to processes in `data` and one to the external mail server), two key-value secrets and a transit key, a database migration with a proof, and dependency, exposure and identity placeholders. `data` has three stateful processes, one volume of each durability class, a sidecar, a configuration file, and a secret placeholder that becomes a reference to a key of a secret. Producing the output of all examples, the seven resolutions included, takes about seven seconds in the build.

Four of the seven oracles, those of `delivery`, `edge`, `observability` and `secrets`, did not exist before this task. They were generated by the production implementation, reviewed, committed, and added to its test suite, so both implementations are tested against them.

4.2.1 Cross-Check Against the Production Implementation

For `auth`, `data`, `delivery` and `minimal`, the script compared these values of every application and process: alert class, secret store, migration, release gate deadline and members, identity, image, user, group, upgrade mode, switchover, deadline, replica count, memory, CPU, hardening, token mounting, eligible machines, bound machine, environment variables with their secret references, writable paths, secrets with their paths and restart targets, volumes with their backup plans, configuration file names, sidecars, and the startup probe. All values were equal.

Two values are computed by both implementations but differ by design. The revision and the input digests are hashes over each implementation’s own form of the model, and the two forms differ. Each implementation is consistent with itself, and only the production implementation’s digests reach the cluster. The configuration file’s name also contains a digest, and there the transformation hashes the text as the production implementation does, because the name appears in the generated files.

4.2.2 Defects Found by Testing

The tests found defects that the examples alone would not have shown. Writing the interface tests showed that a process that runs once, a *job*, was counted as a blue/green process, so a project of only jobs stopped with a release-gate error. A review against the production implementation found that two routes to one port admitted the same proxy twice, that two exposures on one entry point produced two index files with the same path, that the startup probe had no timeout, and that a memory amount written in bytes was misread. Each was fixed, and each has a test or a stop that would catch it again.

4.3 Limitations

- **The knowledge example is not resolved.** The production implementation rejects it, because its images are not in the lock, and its committed edge oracle names processes that the **data** example has since renamed. It will be added when the production implementation resolves it.
- **Some generated files have no entry yet.** The target metamodel describes namespaces, indexes, workloads, switch definitions, service accounts, metrics definitions, network policies and routes. It does not yet describe volume claims, backup jobs, configuration maps, services, disruption budgets, migration jobs or the Vault and secret-operator objects. Task 3 adds them with the templates, so the generated-file entries of **auth** and **data** are incomplete today. Their resolved values, which the files will be generated from, are complete.
- **Two places are less detailed than the production model.** The order between reconcile units is computed per project, as the union over its applications, because Flux orders whole units. A network peer does not record which rule admitted it, so two rules that admit the same peer become one entry, which admits the same traffic.
- **The workload strategy of the delivery tools differs.** For a process that switches by rolling update, the transformation derives a rolling update, while the production implementation's file generator currently writes a recreate strategy. The specification describes a rolling update. The difference will be resolved when the delivery tools are generated.
- **Checks across documents are stops, not diagnostics.** An image missing from the lock stops the run with one message. A checker would list every such problem with its location in the document, as the constraints of Task 1 do.
- **The Eclipse launch is a convenience.** The build runs the transformation through the standalone executor. The launch configuration and the library registration for Eclipse are not run by the build.

4.4 Reflection

QVTo suited this transformation. Most rules are queries over the whole cluster, which OCL expresses well, and the explicit trace made every reference between created elements visible in the code. Three properties of the language cost time: **resolve**, **from** and **access** are keywords and cannot be used as names, a multi-statement **if** needs the brace form, and inside an object expression **result** still refers to the enclosing mapping's result, not to the object being created. The compiler reports each of these clearly.

Testing against another implementation's oracles worked well. The dependency edges were a narrow enough contract to agree on, and the cross-check of the other values found no difference that the edges had hidden. The hand-written target model of Task 1 was also useful: it fixed what the transformation had to produce for **minimal** before any rule was written. The main cost of the approach is that every change to the target metamodel changes that model too.

A Changes to the Task 1 Metamodels

Writing the transformation changed the Task 1 metamodels in three ways. A third metamodel, the pinned inputs, was added (Section A.1). The target metamodel gained the values the transformation derives that Task 1 had not yet recorded, and its generated-file classes now cover applications with several processes (Section A.2). Four grammars were added for the pinned inputs (Section A.3). The source metamodel did not change.

A.1 The Pinned Inputs Metamodel

The package `pinnedinputs` contains the documents the resolution reads beside the source models. They are a separate package because they are not part of what an author writes. The source metamodel’s structure is compared with the production implementation’s schemas through a committed descriptor, and adding these classes to it would have added classes those schemas do not contain. No class of the target metamodel refers to them: the resolved model records the values the transformation reads from them, and their digests. Table 6 lists the classes.

Class	Features	Read by
<code>NodeContract</code>	<code>cluster</code> , <code>label prefix</code> , <code>nodes</code>	eligibility
<code>Node</code>	<code>name (ID)</code> , <code>site</code> , <code>architecture</code> , <code>allocatable</code> , <code>capabilities</code> , <code>gpus</code> , <code>disks</code>	eligibility
<code>Allocatable</code>	<code>cpu</code> , <code>memory</code> , as quantities such as 53500m	eligibility
<code>Gpu</code>	<code>vendor</code> , <code>model</code> , <code>class</code> , <code>memory in MiB</code>	GPU requests
<code>Disk</code>	<code>medium</code> , <code>usable GiB</code>	disk requests
<code>ImagesLock</code>	<code>name</code> , <code>images</code>	every image
<code>LockedImage</code>	<code>alias (ID)</code> , <code>repository</code> , <code>digest</code> , <code>user</code> , <code>group</code>	images, sidecars, backups, migrations
<code>ClusterState</code>	<code>cluster</code> , <code>capture time</code> , <code>bindings</code> , <code>placements</code>	bound machines
<code>Binding</code>	<code>claim</code> , <code>node</code>	bound machines
<code>CurrentPlacement</code>	<code>process</code> , <code>node</code>	not yet read
<code>MigrationProof</code>	<code>applications</code>	migrations
<code>ProvenApplication</code>	<code>application id</code> , <code>tested schema version</code> (optional), whether the release holds a change that cannot run in a transaction	migrations
<code>AssetFile</code>	<code>project</code> , <code>path</code> , <code>text</code>	configuration files

Table 6: The Classes of the Pinned Inputs Metamodel

The disk medium of a node is a string, not the source metamodel’s `Media` enumeration. The node contract may list a medium that no process may request: three machines boot from SD cards, and `sdcard` is not a `Media` literal. `AssetFile` is not parsed from YAML. The pipeline creates one per configuration file that a project names, from the file beside the project document.

A.2 Changes to the Target Metamodel

Table 7 lists the changes. The first four record values the transformation derives. The last three let one generated file describe an application with several processes, which the Task 1 model of `minimal` did not need.

The hand-written model of `minimal` follows these changes. Its startup probe now has a timeout, which changed its revision digest, and its generated files refer to the process through the new features.

A.3 Grammars for the Pinned Inputs

Four grammars were added beside the project and platform grammars: `NodeContract`, `ImagesLock`, `ClusterState` and `MigrationProof`. Each inherits the project grammar, so it reuses its terminals, its indentation tokens and its scalar rule, and adds only the rules for its own document and the keywords it introduces. Listing 7 shows the smallest.

```
grammar dev.jorisjonkers.deploykit.emf.syntax.ClusterState
  with dev.jorisjonkers.deploykit.emf.syntax.ProjectIntent
import "https://jorisjonkers.dev/deploy-kit/pinned-inputs/1"

ClusterState returns ClusterState:
  (('apiVersion' ':' apiVersion=Text)
  & ('kind' ':' kind=Text)
```

Class	Change	Reason
StartupProbe	added <code>timeout</code>	The specification gives the startup probe the platform's probe timeout.
ResolvedGrant	<code>path</code> and <code>access</code> optional; added <code>engine</code> and <code>paths</code>	A transit or database secret has no single path. It is recorded as the Vault paths its policy covers.
PolicyPath	new: <code>path</code> , <code>allows</code>	One path of a secret's policy and the operations allowed on it.
ResolvedVolume	added <code>mountAt</code>	The workload template needs the path a volume is mounted at.
WorkloadFile, SwitchoverFile, IdentityFile	<code>process</code> replaced by <code>processes</code> (one or more)	One file per application holds the resources of all its processes, as in the generated tree of the specification.
NetworkPolicyFile	<code>process</code> , <code>ingress</code> and <code>egress</code> replaced by <code>policies</code>	One policy file per application holds one policy per process. The project's default-deny file holds none.
ProcessPolicy	new: <code>process</code> , <code>ingress</code> , <code>egress</code>	The peers that may reach one process, and those it may reach.

Table 7: Changes to the Target Metamodel

```
& ('schemaVersion' ':' schemaVersion=Text)
& ('cluster' ':' cluster=Text)
& ('capturedAt' ':' capturedAt=Text)
& ('bindings' ':' BEGIN (DASH BEGIN bindings+=Binding END)* END)
& ('placements' ':' BEGIN (DASH BEGIN placements+=CurrentPlacement END)* END));
```

Example 7: The Grammar of the Cluster-State Snapshot

As with the platform document, the file name selects the grammar: `node-contract.yml`, `images.lock.yml`, `cluster-state.yml` and `migration-proof.yml`. One input file changed. The scalar terminal of the project grammar cannot contain a colon, so the snapshot's capture time is now quoted, as digests and URLs already were. The production implementation reads the same value either way.

B The Hand-in Archive

The archive holds this report as `report.pdf`, a `README.md` with the steps below, and two folders, laid out as Table 8 shows. The first, `task-2-transformations/`, is an Eclipse project for reading the models, imported with File > Import > Existing Projects into Workspace. The second, `implementation/`, contains the EMF implementation and the examples its tests read.

Path	Contents
<code>model/</code>	The source, pinned-inputs and target metamodels.
<code>transforms/</code>	<code>lower.qvto</code> and <code>resolution.qvto</code> .
<code>examples/platform/</code>	The platform document and the pinned inputs every example reads.
<code>examples/<example>/authored/</code>	The project document with its env and configuration files.
<code>examples/<example>/resolution.intent.xmi</code>	The intent extent: authored and lowered projects, and the platform document.
<code>examples/<example>/resolution.pinned.xmi</code>	The pinned extent.
<code>examples/<example>/<project>.resolveddeployment</code>	The resolved model the transformation wrote.
<code>examples/<example>/dependencies.json</code>	The edges exported from it, and <code>expected-dependencies.json</code> , the oracle.
<code>examples/minimal/minimal.hand-written.resolveddeployment</code>	The hand-written target model of Task 1, as changed in Appendix A.2.

Table 8: The Models Project of the Archive

Every XMI declares its metamodel in `xsi:schemaLocation`, so Eclipse opens it in the Sample Reflective Ecore Model Editor without a registered package. The models project contains no Java, so the transformation does not run from it: the resolution calls the `Hashing` library, which the implementation's plug-in registers.

`./mvnw clean verify` in `implementation/emf` builds the implementation and runs every test; it needs JDK 21. It parses every example, runs both transformations, writes each example's output under `bundles/cli/target/parity/<example>/`, and runs the parity test that compares each `dependencies.json` with its oracle. To run the transformation in Eclipse Modeling Tools with the QVT-Operational SDK, import `implementation/emf` as existing Maven projects after that build, and run `resolution.launch` in `dev.jorisjonkers.deploykit.emf.resolve`. It resolves `minimal` from the extents the build wrote.